



Worksheet 6

Student Name: Aryan
Branch: MCA (AI&ML)
Semester: 2nd
Subject Name: DBMS LAB

UID: 25MCI10082
Section/Group: 1/A
Date of Performance: 01/03/2026
Subject Code: 25CAP-652

1. Aim of the Session

Learn how to create, query, and manage views in SQL to simplify database queries and provide a layer of abstraction for end-users.

2. Software Requirements

- PostgreSQL (Database Server)
- pgAdmin
- Windows Operating System

3. Objective of the Session

- **Data Abstraction:** To understand how to hide complex table joins and calculations behind a simple virtual table interface.
- **Enhanced Security:** To learn how to restrict user access to sensitive columns by providing views instead of direct table access.
- **Query Simplification:** To master the creation of views that pre-join multiple tables, making reporting easier for non-technical users.
- **View Management:** To understand the syntax for creating, altering, and dropping views, as well as the naming conventions required for efficient data access.

4. Practical / Experiment Steps

Prerequisite Understanding

- Students should first understand that a View is a virtual table created from a SELECT query and does not store data physically.
- Students should understand the difference between base tables (which store data) and views (which display data dynamically from base tables).
- Students should create at least two related tables (for example, Employee and Department) to demonstrate simple and complex view creation.
- The base tables should contain:
 - A unique identifier (Primary Key)
 - Related columns for establishing relationships (Foreign Key)
 - Descriptive attributes such as name, salary, or status
- Students should understand the concept of JOIN operations before creating multi-table views.
- Students should understand aggregate functions (COUNT, SUM, AVG) for creating summarized views.
- Students should be familiar with basic SQL commands including:
 - CREATE VIEW
 - SELECT
 - CREATE OR REPLACE VIEW
 - DROP VIEW
- Students should understand that views can be used as a security layer by granting access to the view instead of the original tables.

5. Procedure of the Practical

- (i) Start the system and log in to the computer.
- (ii) Open PostgreSQL software.

iii) Create and select the database.

Create database Practical6;

(iv) Create table using DDL command.

```
create table Department (  
    dept_id serial primary key,  
    dept_name varchar(50)  
);
```

```
create table EmployeeV (  
    emp_id serial primary key,  
    emp_name varchar(50),  
    salary numeric(10,2),  
    status varchar(20),  
    dept_id int references Department(dept_id)  
);
```

(v) Insert records into the table.

```
insert into Department (dept_name) values  
  
('HR'),  
  
('IT'),  
  
('Finance'),  
  
('Sales');
```

```
insert into EmployeeV (emp_name, salary, status, dept_id) values
```

('Amit', 30000, 'Active', 1),
('Ankit', 45000, 'Active', 2),
('Khushi', 58000, 'Inactive', 2),
('Manisha', 40000, 'Inactive', 3),
('Arman', 70000, 'Active', 4);

(vi) Display all records.

select * from EmployeeV;

	emp_id [PK] integer	emp_name character varying (50)	salary numeric (10,2)	status character varying (20)	dept_id integer
1	1	Amit	30000.00	Active	1
2	2	Ankit	45000.00	Active	2
3	3	Khushi	58000.00	Inactive	2
4	4	Manisha	40000.00	Inactive	3
5	5	Arman	70000.00	Active	4

select * from Department;

	dept_id [PK] integer	dept_name character varying (50)
1	1	HR
2	2	IT
3	3	Finance
4	4	Sales

Step 1: Creating a Simple View for Data Filtering

create view active_emp as

select emp_id, emp_name, salary, dept_id

from EmployeeV

where status = 'Active';

select * from active_emp;

	emp_id integer	emp_name character varying (50)	salary numeric (10,2)	dept_id integer
1	1	Amit	30000.00	1
2	2	Ankit	45000.00	2
3	5	Arman	70000.00	4

Step 2: Creating a View for Joining Multiple Tables

create view

emp_dept_view as

select

e.emp_id,

e.emp_name,

e.salary,

```
d.dept_name  
  
from EmployeeV e  
  
join Department d  
  
on e.dept_id = d.dept_id;  
  
select * from  
emp_dept_view;
```

	emp_id integer 🔒	emp_name character varying (50) 🔒	salary numeric (10,2) 🔒	dept_name character varying (50) 🔒
1	1	Amit	30000.00	HR
2	2	Ankit	45000.00	IT
3	3	Khushi	58000.00	IT
4	4	Manisha	40000.00	Finance
5	5	Arman	70000.00	Sales

Step 3: Advanced Summarization View

```
create view dept_summary as  
  
select  
  
d.dept_name,  
  
count(e.emp_id) as total_employees,  
  
avg(e.salary) as average_salary,  
  
sum(e.salary) as total_salary
```

from Department d

left join EmployeeV e

on d.dept_id = e.dept_id

group by d.dept_name;

select * from dept_summary;

	dept_name character varying (50) 🔒	total_employees bigint 🔒	average_salary numeric 🔒	total_salary numeric 🔒
1	Finance	1	40000.00000000000000	40000.00
2	Sales	1	70000.00000000000000	70000.00
3	IT	2	51500.00000000000000	103000.00
4	HR	1	30000.00000000000000	30000.00

create or replace view active_employees as

select emp_id, emp_name, salary

from EmployeeV

where status = 'Active';

select * from active_employees;

	emp_id integer 🔒	emp_name character varying (50) 🔒	salary numeric (10,2) 🔒
1	1	Amit	30000.00
2	2	Ankit	45000.00
3	5	Arman	70000.00

6. I/O Analysis (Input / Output) Input:

- Creation of base tables such as Employee and Department
- Insertion of sample records into base tables
- SQL queries used to create simple and complex views
- JOIN operations applied within view definitions
- Aggregate functions (COUNT, SUM, AVG) used in summary views **Output:**

- Virtual tables (Views) created successfully without storing physical data
- Filtered employee data displayed through simple views
- Combined employee-department data retrieved using JOIN-based views
- Department-level statistical summaries generated automatically
- Simplified query results for reporting purposes
- Secure and controlled data access provided through view-based permissions
- Real-time updated results when underlying base tables are modified

Screenshots of execution and obtained results are attached.

7. Learning Outcomes

- **Abstraction Proficiency:** Students will be able to create and query views to simplify efficient data access and abstraction.
- **Security Implementation:** Students will understand how to use views for data masking and providing restricted access to sensitive information.
- **Syntactic Accuracy:** Students will demonstrate the correct syntax for creating and querying views, ensuring logical clarity in naming conventions.
- **Real-world Application:** Students will be able to design views for practical domains like Library Management Systems or Payroll Systems to demonstrate functionality.